Solitito — Project Summary

A real-time guitar chord recognition system

Version 0.5.8, October 2026

---

1. System overview

Solitito is a real-time guitar trainer implemented in Rust. The program takes a signal from a microphone or audio interface, recognises the material being played, and guides the user through jazz standards, intervals, scales, arpeggios, interval formulas and the layout of the neck.

Recognition is performed by a neural network of 7.3 million parameters exported to the ONNX format. A second, small network of 260 thousand parameters answers one narrower question on every 16 ms of audio: which pitch class was just struck. It decides when a note counts again (8.13). All processing — DSP, inference and the user interface — is carried out locally on the CPU, without a network connection and without external services.

Six modes of operation are provided:

- **Chords** — complete jazz standards. Green indicates an exact match, yellow a triad or a common substitution, red a chord that was recognised but at a signal level too low to confirm.
- **Intervals** — chord tones played individually, with a selectable set of degrees to practise.
- **Scales** — sequential traversal of the notes defined by a scale.
- **Arpeggios** — chord tones in sequence over a given progression.
- **Fretboard** — a region of the neck is drawn at random, comprising a set of strings and a span of four frets, and then held; the user is asked for successive notes lying within it. The mode serves to learn the positions of the notes within a single hand position.
- **Formulas** — a set of intervals drawn over a root and played in any order, with the option of planting that same set on a chord or carrying it across the chords of a standard. The mode is described in 8.11.

Work on the project began in December 2025. This document presents the system architecture, the course of the work, and the design decisions together with their justification.

---

2. Methodology

The project adopted the principle that every change requires justification by measurement rather than by hypothesis. In practice this meant developing a set of **probes** — scripts that answer a single question at low computational cost, without requiring retraining.

probe	question addressed
`verify_annotations.py`	do the labels describe the audio content?
`probe_root.py`	how often does the labelled root actually sound within the window?
`probe_sources.py`	which GuitarSet chord annotation is usable?
`probe_quality.py`	from which source should chord quality be derived?
`inspect_jams.py`	what do the JAMS files actually contain?
`latency_material.py`	generates plucks whose onsets are known exactly, as a yardstick
`latency_ground_truth.py`	extracts onsets and pitches from a REAL recording, for the same measurement
`latency_stats.py`	how late does the application learn what was played, and how often wrongly?
`latency_rules.py`	what would each crediting rule cost on that recording?

The first five address the data and the model; the remaining four form a chain and are used together. Material is prepared — either synthesised, with onsets known by construction, or taken from a recording of real playing — the file is then passed through the application's own feature path with `./solitito --probe file.wav --step 1`, and the resulting listing is read by the last two scripts. `latency_stats.py` separates the three answers the application can hold: the model's pitch head, the single-frame estimate and, in listings made before 0.5.10, the onset head the model carried until then. `latency_rules.py` replays the crediting rules over the same listing and reports, for each, how many credits it would hand out that nobody played and how many notes it would miss altogether. The table in 8.11 comes from that script.

Two further tools are not probes but belong to the same set: `gen_weights.py`, which produces the sparse CQT kernel shared by the trainer and the application, and `gp5_to_arpeggio.py`, which converts a Guitar Pro file into the degree notation the Arpeggios mode reads. `hf_cleanup.py` clears the checkpoint repository before a run begins from scratch. `model_trainer.py` trains the chord network and `strike_trainer.py` the strike detector (6.6); `extract_onset_branch.py` cuts the strike detector out of a combined take7 model.

Two more instruments now serve the crediting rules. The application can record a session together with a journal of every judgement it made (`SOLITITO_RECORD`), so a false credit reported from practice can be replayed and explained. And a harness inside the test suite splices the 51 reviewed notes of one labelled recording into new signals — a note ringing out, the same note struck again, a note ringing into the next chord — and runs the real judge over them. The tables in 8.13 come from it.

The methodology proved effective on repeated occasions. Its converse should also be noted: **hypotheses formulated prior to measurement proved to be wrong in a systematic manner.** These cases are catalogued in Chapter 9.

3. The synthetic dataset

The script `dataset_generator_v2.py` produces, in a single pass, a Guitar Pro file **together with** a complete set of annotations.

3.1. Contents

394 blocks of 6 seconds each (3 bars at 120 BPM), comprising:

- 12 roots × {maj, min, maj7, dom7, min7, m7b5, dim7, sus4, aug} in several positions on the fretboard,
- all 96 single notes (6 strings × 16 frets).

Block structure: the first bar constitutes the attack, the second the sustain (tie), and the third silence. The annotation covers the interval `[start + 0.05 s, +3.2 s]`, that is the attack together with the sustain, excluding the decay tail.

3.2. Verification of chord shapes

Before generation begins, the script checks **each of the 21 movable shapes at every fret**, verifying that the shape genuinely produces the declared intervals. An error in the shape table halts generation rather than propagating into the dataset.

3.3. Rendering

The guitar track is exported as a DI signal and rendered in a DAW using NAM in two variants:

- `synth_dataset_clean.wav` — Fender Deluxe Reverb, clean tone,
- `synth_dataset_eob.wav` — at the edge of breakup.

The recommended sample rate is **48 kHz**, for two independent reasons. NAM operates natively at 48 kHz, so at 44.1 kHz the plugin performs internal resampling. Furthermore, decimation to the 16 kHz at which the model operates is exact at 48 kHz (`48000/16000 = 3`) and is not exact at 44.1 kHz (`2.75625`).

3.4. Calibration and verification

The `--calibrate <wav>` mode determines the position of the first attack, in case the DAW has added silence at the beginning of the file. PCM 16/24/32-bit and floating-point 32/64-bit formats are supported, in mono and stereo, by means of a self-contained WAV reader with no external dependencies.

The script `verify_annotations.py` compares the label against the **actual audio content**: the dominant pitch class within the window is set against the root indicated by the label. Its only dependency is numpy.

Reference values obtained from the v2 render:

	clean	eob	random baseline
top1	87%	77%	~8%
top3	**100%**	**98%**	~25%

The top3 figure is the meaningful one, since within a chord the root is frequently quieter than the third or the fifth. A value of `top3 > 75%` qualifies the dataset for training; `top3 ≈ 25%` indicates that the labels do not describe the signal.

The script additionally verifies the block period derived from the energy envelope, together with the offset between the start of the annotation and the attack. The two tests are **complementary**: an annotation shifted by an integer multiple of the block length coincides with the attack of an adjacent block and remains undetectable by the timing test. Only a comparison of the labels against the signal content will reveal it.

An overriding principle was adopted: **the generator writes labels directly, and an independent script verifies their agreement with the audio.** No processing step reconstructs labels from the signal.

---

4. The GuitarSet dataset

GuitarSet comprises 360 recordings with annotations in JAMS format, captured with a hexaphonic pickup. It is the only source of material from a real instrument used in the project. Bringing it into a usable form required four training runs.

Four properties of the dataset are described below, the omission of which reduces model accuracy.

4.1. Half of the dataset contains no chords

Every excerpt was recorded twice: as `_comp` (accompaniment) and as `_solo` (monophonic improvisation). **The chord annotation is identical in both cases** — it describes the progression over which the performer improvised.

Training the chord heads on solo files amounts to teaching the model that a single note constitutes a complete jazz chord. The solo material comprises 180 of the 360 files.

Filtering these files moved the `Exact` figure from **44.8% to 82.3%** in a single run.

The **pitch** targets derived from solo files remain entirely correct — this is genuine monophonic playing with accurate note annotations, and therefore material well suited to a single-note detector. The trainer consequently retains the pitch loss on solo windows and masks the root and quality only (`GUITARSET_SOLO_MODE = "mask_chord"`).

An indirect consequence should be noted. The `probe_root.py` probe initially determined root audibility across all 360 files, yielding a ceiling of 64.1%. On this basis it was concluded that jazz performers employ rootless voicings and that the chord name is not a function of the signal. That conclusion proved incorrect: once the solo material was filtered out, the root sounds in **97%** of accompaniment windows.

4.2. Chord annotations exist in two variants

Every file carries an `instructed` annotation (the chord as written) and a `performed` annotation (a transcription of the performance). The number of segments is identical; the distribution of labels is not:

quality	instructed	performed	difference
maj	2640	2106	−534
min	960	460	**−500**
min7	**0**	**360**	**+360**
maj7	**0**	**430**	**+430**
dom7	480	694	+214
m7b5	240	134	−106
sus	0	132	+132
total	**4320**	**4320**	**0**

The segment total is unchanged, and the choice is therefore not one between more and less data, but a **relabelling of the same recordings**.

The `min` and `min7` rows must be read together: **five hundred segments written as `m` were performed as `m7`.** Training on the `instructed` annotation teaches the model to classify a voicing containing a minor seventh as a plain minor chord. This error was subsequently observed in the application, where the chord `Gm7` was recognised as `Gm`.

Furthermore, the `instructed` annotation contains **not a single** instance of the `maj7` and `min7` classes. Until the switch was made, both classes originated exclusively from the two renders of the synthetic dataset, that is from a single instrument processed through a single amplifier. This produced a figure of 100% on the validation set (the same instrument on both sides of the split) together with an absence of robustness on real material.

Switching to the `performed` annotation moved the `Exact` figure from **82.3% to 92.4%**. Root accuracy was unaffected: the two annotations differ with respect to the root in **0 out of 43,056** comparisons.

4.3. The split must follow the source

Randomly shuffling the list of chord segments and splitting it 94/6 places adjacent bars of **the same recording** on both sides of the split — with the same instrument, room, microphone and take, and frequently with the same chord recurring one bar later.

In the synthetic dataset the dependency is stronger still: the `clean` and `eob` renders of one block constitute the same performance processed through a different amplifier, and were assigned to the training and validation sets independently.

The solution adopted is grouping by source: the entire file for GuitarSet and the entire block (both renders) for the synthetic dataset.

All validation figures fall following this change. This does not constitute a regression of the model, but the removal of an inflation that invalidated earlier conclusions regarding generalisation. The value `root_acc = 98%` obtained in run take1 was to a large extent an artefact: with `both` annotations the same segment occurred twice, so an identical window entered both the training and the validation set.

4.4. Pitch targets derived from `note_midi`

The chord annotation describes the **intended harmony** over a span of several seconds. The training window covers 0.77 s and frequently does not contain the labelled seventh at all. The model was therefore penalised for failing to predict a note absent from the signal — seventh recall on GuitarSet stood at 32%.

The hexaphonic pickup provides `note_midi` annotations describing the actual performance on each string separately. Deriving the pitch targets from these raised seventh recall from **32% to 96%**.

The threshold adopted: a note must sound for at least 25% of the window (`NOTE_MIN_COVER`) in order to be included in the target.

---

5. Architecture

5.1. Signal path

```
audio input → resampling to 16 kHz → FFT (8192) → sparse pseudo-CQT → features → ONNX model
```

Resampling to 16 kHz. The CQT spans 6 octaves from C1, so the highest bin falls in the region of 2 kHz, well below the 8 kHz Nyquist limit. Bandwidth does not constitute a constraint.

Pseudo-CQT. In place of a true constant-Q transform, the application multiplies the FFT spectrum by a precomputed kernel: 144 bins, 24 per octave, corresponding to quarter-tone resolution. The kernel is obtained from `librosa.filters.constant_q`, by which means the application and the trainer produce identical features.

Features. 168 values per frame:

range	contents
0–143	CQT bins after log normalisation
144–155	chroma (`cq_to_chroma` matrix, per-frame maximum normalisation)
156–167	bass energy (mean of bin pairs 0–23)

The model spans 48 frames of history at a hop of 256 samples, corresponding to **0.77 s**.

5.2. Network

```
input [48, 168]
   ↓
InstanceNorm2d
   ↓
ConvBlockSE  1 → 48       (Squeeze-and-Excitation)
ConvBlockSE 48 → 96
ConvBlockSE 96 → 192
ConvBlockSE 192 → 384
   ↓
Linear 3840 → 384
   ↓
+ CLS token, positional encoding
   ↓
TransformerEncoder: 4 layers, 8 heads, d=384, FF=768, GELU, norm_first
   ↓
CLS
   ├── fc_root     → 13   (12 pitch classes + "Noise")
   ├── fc_quality  → 11   (maj, min, maj7, dom7, min7, m7b5, dim7, aug, sus, note, N)
   └── fc_pitch    → 12   (sigmoid: which pitch classes are sounding)
```

Total parameter count: 7,286,038. This is `best_model_v2_take6_chords.onnx`, the file the application loads.

Until 0.5.10 the network carried a fourth head, `fc_onset` (156,156 parameters, trained in phase 4, 6.1): which pitch classes were STRUCK. It did not read the CLS token. Its input was assembled from four parts — the encoder's last frame, the difference between that frame and the one six frames earlier, and, taken from the raw features before the encoder sees them, the RISE of the CQT folded onto pitch classes together with the rise of the chroma: an attack adds energy to the

spectrum and a decay does not. The strike detector (5.4) answers the same question better, and replaced it everywhere in the judge; the file the application loads is exported without the head, and its three outputs are bit for bit those of the four-head file.

5.3. Division of tasks between the heads

The distinction between the roles of the individual heads is of central importance and has been confirmed by measurement.

head	result	role
`pitch_logits`	F1 0.909	which notes are sounding — the basis of the Intervals, Scales, Arpeggios and Fretboard modes
`root_logits`	98.1%	the name of the root
`quality_logits`	~93%	the chord family

An early version of the application derived chord quality from the pitch vector using manually determined thresholds. The `probe_quality.py` probe compared three methods on the same checkpoint:

method	accuracy
the `quality_logits` head	**80.5%**
template matching against the predicted pitch	66.0%
template matching against the **true** pitch vector	59.2%

The head exceeds template matching against a *precisely known* set of notes by 21 percentage points. It therefore extracts from the signal information not present in the set of pitch classes itself: timbre, the distribution of the voicing across the register, and the shape of the attack.

Design conclusion: the quality head remains a necessary component.

None of the three answers what was STRUCK. "Sounding" is true of a string ringing on, of one resonating in sympathy, and of the note played immediately before — the model's window is 0.77 s wide and holds all of them. The former onset head was built for that question; measured against a recording it was the fastest answer the application had (202 ms after the strike, against 676 ms), but the least precise as to WHICH string was struck, since an attack spreads onto the neighbouring ones. The strike detector (5.4) took its place.

5.4. The strike detector

The former onset head shared the recognition model's window and its pace: 0.77 s of audio, asked every 40 ms. That is too coarse to tell whether a note that is already ringing was just struck again, which is the question repeated notes pose. The strike detector is a separate network built for that question alone:

```
two Hann windows of the newest audio, 1024 and 2048 samples (64 and 128 ms)
   ↓  magnitudes up to 4 kHz, log1p(1000·a)/log(1001): 770 features per 16 ms hop
Conv1d 770 → 96  +  Conv1d 770 → 96 on the RISE of each bin over the four frames before
   ↓
4 × causal residual Conv1d 96 → 96, kernel 3, dilations 1, 2, 4, 8
   ↓
Conv1d 96 → 12   (sigmoid: which pitch classes were struck at this frame)
```

It is causal and needs 34 past frames plus the current one — 0.56 s. It has 260,076 parameters and costs 0.7 ms per hop on the CPU, so the application asks it on every hop. The rise projection carries the same idea as the former onset head's input: an attack adds energy to the spectrum and a decay does not.

It was trained as a branch of a combined model beside the frozen chord network (6.6). The application loads it cut out of that file, `short_onset_masking_v2.onnx` (1 MB), because the branch never reads the chord input: its answer is identical to the last digit whatever that input holds.

---

6. Training

6.1. Phases

The training procedure comprises four phases, of which the first is of principal significance.

phase	scope	status
1	main training, 120 epochs, cosine LR with warm-up	the only phase yielding improvement
2	tuning of the pitch head threshold	corrected — it had sorted by a metric independent of the threshold
3	head fine-tuning with the encoder frozen	**disabled**
4	the onset head, trained alone	trained, no longer shipped: since 0.5.10 the export leaves it out

Phase 4 trains `fc_onset` with every other parameter frozen. The construction is deliberate: the phase either yields a head worth using or leaves the model exactly as it was, and the three earlier outputs cannot move by a decimal place. Its material is the note-level annotation — windows sampled around real attacks, by default from the solo recordings alone, which are the ones carrying `note_midi`. Training ends with a threshold sweep reported as F1. The head is not in the file the application loads: after phase 4 the trainer exports the chord network without it, `best_model_v2_take6_chords.onnx`, and the judge learns what was struck from the strike detector (5.4).

Phase 2 scanned thresholds over the range 0.30–0.70, optimising the `exact` figure. That figure is the conjunction of `argmax(root)` and `argmax(quality)`, and therefore took an identical value across all 41 thresholds, making the selection arbitrary. Sorting is now performed by the F1 of the pitch head, on which the threshold genuinely bears.

Phase 3 was disabled after its effects were measured over three consecutive runs:

```
take2, 40 epochs: pitch_f1 0.9318 → 0.9326 (+0.0008), exact 0.5455 → 0.5445
take3,  4 epochs: F1 0.933 → 0.931,             exact 54.6% unchanged
```

The encoder remains frozen and only the heads are trained, at a learning rate of 1e-5. The phase possesses no mechanism by which to improve the model, and its cost amounts to approximately 1.5 hours of computation.

6.2. Loss functions and masking

- **root** — CrossEntropy with label smoothing of 0.05,
- **quality** — CrossEntropy with smoothing, sampler weighted by class,
- **pitch** — Focal BCE ($\gamma = 2.0$, `pos_weight` 2.5), auxiliary weight 0.7.

Two masking mechanisms were applied, both justified by measurement.

`MASK_ROOT_WHEN_SILENT` — the root loss is computed exclusively on windows in which the root genuinely sounds. Training the root on windows in which it is absent does not result in learned perception, but in memorisation of the GuitarSet progressions, while the shared encoder receives a gradient in conflict with the pitch target.

`GUITARSET_SOLO_MODE = "mask_chord"` — the root and quality receive no gradient from solo recordings; the pitch head receives it unchanged.

6.3. Augmentation

- **pitch shift** by ±N semitones. An implementation detail of consequence: the CQT and the bass energy are shifted **with zero fill**, whereas the chroma is shifted **cyclically**. The chroma is circular by definition; the CQT is not, and wrapping the bass band around to the top of the range would introduce notes absent from the signal.
- **time and frequency masking** (SpecAugment),
- **spectral tilt and noise** — simulation of varied signal chains.

6.4. Energy gate

The parameter `ENERGY_KEEP_FRAC = 0.55` rejects windows whose energy falls below 55% of the segment peak. The justification: during the decay phase the seventh, being the quietest component of the voicing, is the first to disappear, while the label remains unchanged. In the absence of the gate this would systematically train the collapse `m7 → m`.

6.5. Metrics

Chord metrics are computed **exclusively on windows in which the label describes the signal**, with solo windows excluded. They are additionally reported separately for windows with and without an audible root, since the combined figure conflates two distinct populations.

Selection of the best checkpoint proceeds by the figure `composite = (root_audible + qual + exact) / 3`. Use of the combined `root_acc` would favour a model that reproduces progressions effectively over one that analyses the signal correctly.

The `TRAIN` diagnostic check, performed every 5 epochs, computes metrics on the training data without augmentation. It addresses the question of whether the model is capable of reproducing its own training data. A negative answer identifies the features or the labels as the source of the constraint rather than generalisation, and indicates that increasing the number of epochs would serve no purpose.

6.6. The strike detector

The strike detector is trained apart from the chord network, by a trainer of its own: the two networks share no weight and no input, only the application. Its material is labelled by attack rather than by chord:

- **GuitarSet** whole takes, solo and comp, with players split whole — 00 to 03 for training, 04 for validation, 05 for test. The note starts are annotations, not verified pick attacks.
- **Synthetic plucks** (Karplus–Strong), whose attack times are exact by construction: notes held, the same note plucked again while it rings, octaves, a third or fifth added over a ringing root, triads re-strummed.

- **Masking pairs**: a root and third ringing, then an upper fifth struck at −18, −12, −6 or 0 dB relative to them, measured over its first 96 ms. Each comes in three variants sharing background and gain — background alone, the fifth alone, both together — so the only difference between them is the strike.

Training levels vary by ±6 dB. The epoch is chosen on validation at a fixed threshold of 0.5, the threshold afterwards on the exported ONNX — what the application will actually run — and the test split is scored once, at that threshold. The released model came out of three runs, each fine-tuned from the previous one's best weights: Rise from scratch, then with the chord base frozen (`v2_take7`), then with the masking pairs (`v2_take7_masking_v2`, threshold 0.9).

The strike detector's trainer is one file, `dist/strike_trainer.py`, that can be pasted whole into a notebook. It replaced a dozen experiment scripts and three generators that assembled them; those experiments remain in the `rise` branch. The merge was verified rather than assumed: on CPU the one file trains bit for bit like the scripts the released model was trained with — the same data, batches, losses, weights, threshold and ONNX outputs, from scratch and from a parent checkpoint — and a rerun of the released recipe on a Kaggle GPU arrived at the same weights. It writes the application's file straight from the trained network; from the released checkpoint that file answers bit for bit like the released `short_onset_masking_v2.onnx`. The chord network keeps its own trainer, `dist/model_trainer.py`, the one that produced `best_model_v2_take6_onset.onnx`: exported from that run's checkpoint, all four outputs match the released file bit for bit. It also writes the three-output file the application loads, `best_model_v2_take6_chords.onnx`, whose outputs are bit for bit the first three of the four-output one. The 770 features are computed twice, by the trainer in Python and by the application in Rust; a shared fixture holds both to the same numbers, bit for bit, from both test suites.

---

7. Results

Model `v2_take6`, validated with a split by source and with solo windows excluded:

metric	value
root accuracy	**98.1%**
pitch F1	**0.909**
exact match (root **and** quality)	**92.4%**
onset F1 (head no longer shipped)	**0.812**

The first three figures are identical in the three-head and four-head files: the onset head was trained with the rest of the network frozen. The fourth figure is taken at the threshold maximising F1 on the validation split; the head is gone from the released model since 0.5.10.

Accuracy by quality at the best checkpoint: `dom7` 97%, `min7` 93%, `min` 92%, `sus` 91%, `maj` 89%, `maj7` 89%; the classes `m7b5`, `dim7` and `aug` above 97%.

7.1. Course of the work

run	change	Exact
take1–take3	various, on a split subject to leakage	not comparable
take4	split by source — reference point	44.8%
take5	masking of solo recordings	82.3%
take6	`performed` annotations	**92.4%**

7.2. Limiting factor

The difference between the training and validation sets with respect to quality amounts to **6.5 percentage points** (99.2% against 92.7%). The model reproduces the training data. This corresponds to the profile of a constraint imposed by **generalisation** rather than by the capacity of the architecture.

Practical conclusion: neither an increase in the number of epochs nor an increase in model size will yield improvement. Improvement will follow from an increase in the quantity of varied material from a real instrument.

8. The application

Recognition accuracy is not equivalent to the usefulness of a trainer. Three matters proved to carry weight comparable to changes in the model.

8.1. Requirement for a full context window

The trainer determined windows **exclusively within** the sustaining chord (`range(start, end - 48)`). Following a strike of the strings, the application buffer contains partial silence for 0.77 s; taking the FFT window into account (8192 samples, that is 512 ms), the oldest frame describes signal from as much as 1.3 s earlier. This constitutes an input outside the training distribution.

The observed symptom: seventh chords were recognised only during the sustain phase, that is at the first moment at which the window becomes entirely filled with the chord.

The solution adopted was a single threshold: the application issued no query to the model until the window was 90% filled with signal. That threshold is correct for the chord NAME and wrong for everything else. Playing one note at a time never fills a window to nine tenths — 43 frames, that is 688 ms of unbroken sound — so the model was not asked at all, the display froze on the last chord it had, and everything downstream froze with it.

The requirement is now split in two. The model is asked from half a window (measured: at 50–70% fill the pitch head names an isolated note correctly in every frame of the measurement), and its chord name is believed only from nine tenths. Both thresholds are stated as constants at one place in the application, because the gate on the name is applied on the far side of the channel from the thread that asks, and written out twice they were free to drift apart.

8.2. Differing decay rates of chord components

Diagnostic output for a sustained `Gm7` chord:

```
G m7 | min7=96% | b7=96      ← immediately after the strike
G m7 | min7=82% | b7=76
G m7 | min7=52% | b7=52
G m  | min=49%  | b7=45      ← the seventh has faded, the model reclassifies
```

The model's classification is correct — the seventh is genuinely no longer present in the current window. A chord does not, however, change its identity in the course of sustaining.

The solution adopted: a quality latch. The mechanism **engages** at a confidence of ≥ 0.60 but **holds its state independently of confidence**. During the decay phase the model reports the reduced quality at a confidence in the region of 94–96%, so a confidence threshold alone would afford no protection. The latch is released by a new attack or by a change of root.

The latch engages only 48 frames after the attack. Before that point the window still contains the tail of the **preceding** chord, which would result in an incorrect name being latched.

Attack detection compares the signal level against a slowly moving envelope (EMA) rather than against an absolute threshold, which would depend on playing volume. A refractory period of 0.2 s was additionally applied so that a single strum triggers exactly one attack.

8.3. Measuring time rather than assuming it

The progress counter received the fixed value `dt = 0.040`, whereas the inference thread required 55–90 ms per cycle (inference together with 40 ms of sleep). The counter therefore ran slower than real time: a threshold of 0.6 s was reached after approximately one second, with the value depending on machine load.

Following correction — comprising measurement of real elapsed time, determination of the period by the inference thread from the start of the cycle, narrowing of the voting window from 5 to 3, and reduction of the default threshold to 0.25 s — the transition takes **approximately 0.3 s** in place of 1.2 s.

8.4. Sparse representation of the CQT kernel

The full kernel comprises 4097 × 144 = 589,968 weights, concentrated around the centre frequency of each bin. Discarding weights below 1e-4 of the peak value yields the following results:

threshold	weights retained	max. error relative to peak
1e-5	21.9%	0.006%
1e-4	**6.9%**	**0.033%**
1e-3	2.3%	0.352%

The error was determined on three spectra: white, pink, and a guitar-like harmonic series. The CQT is followed by log normalisation over an 80 dB range, so a figure of 0.03% remains orders of magnitude below the resolution of the feature.

The size of the weights file falls from **28 MB to 2 MB**, and the audio path performs approximately **14 times fewer multiplications** per frame. The previous implementation traversed all 4097 FFT bins for each of the 144 CQT bins, discarding zero values only within the loop.

8.5. Feature agreement between the trainer and the application

Discrepancies of this class are particularly difficult to diagnose, since the application remains functional and exhibits only classification errors. Two were identified:

- **chroma mapping.** The file distributed with the application folded bins in the pairs `(0,1), (2,3), …`, whereas `librosa.cq_to_chroma` applies the division `(1,2), (3,4), …`. Every second bin was assigned to the adjacent class, which corresponds to a chroma blurred by a semitone across half the band.
- **cache key.** The cache file name was derived from the expression `abs(hash(path))`. The Python interpreter randomises the hash seed for strings on every process start, so the cache was never reused between sessions. An SHA-1 digest of the file name is now employed.

The application **rejects** weights in the previous, dense format, reporting this by means of a message.

8.6. User interface

Following a review, the number of controls was reduced from five to four:

control	remarks
Noise gate	in dBFS, with a level meter on the same scale and a threshold marker

control	remarks
Chord confidence	threshold for the chord name (Chords mode)
Note threshold	threshold for a single note (note-based modes)
Hold time	the period for which a correct chord must be sustained

The `Tail` control (set from the interface and read nowhere in the code) and the `In gain` control (whose effect was cancelled by the per-frame normalisation, so that it shifted only the same inequality as the noise gate) were removed.

The `Confidence` control governed two distinct quantities simultaneously, and in the note-based modes was subject to a lower bound of `.max(0.5)`, as a result of which the entire range 0.1–0.5 produced identical behaviour. The functions were separated.

The noise gate previously operated on a linear RMS scale of 0–0.1, which **did not reach the noise level of a laptop microphone** (RMS 0.05–0.15 after gain). A decibel scale of −72…0 dBFS provides resolution across the required range together with coverage to full scale.

The panel has since grown past what one column can hold and is divided into four tabs — the input and the gate, how strictly what is played is judged, what is to be played, and what the window shows. The third of these holds only what belongs to the mode on screen: a song has nothing to say in Formulas and a formula nothing in Chords, so it is a different tab in each of them. Only one tab is drawn at a time, so there is correspondingly less to redraw while playing.

8.7. Diagnostic mode

```
SOLITITO_DEBUG=1 ./solitito
```

The mode prints, on each prediction, the three strongest qualities together with the pitch vector expressed as **intervals relative to the recognised root**:

```
G m7  | min7=97% sus=0% maj=0% | R96# b25 28 b382# 37 44 b56 594# b616 69 b797# 74
```

The tool distinguishes the case in which the model fails to detect the seventh from the case in which it detects it but disregards it in classification. Both symptoms are indistinguishable at the level of the chord name and call for opposite corrective actions. The `Gm7` case was resolved by these means without retraining.

8.8. Single notes are not a question the model can answer

The model is asked about 48 frames — 0.77 s — and answers about all of it. That is correct for a chord held under the fingers and wrong for a scale, where notes follow one another faster than the window empties.

Measured with `--probe` on a scale at 0.6 s per note, the rule then in force (the target class above threshold and within 10% of the loudest) credited:

	the note being played	**the note before it**
the model's pitch head	7% of windows	79%
a single CQT frame	57%	43%

The model is not at fault: on isolated sustained notes it places 0.96–0.99 on the correct pitch class, and on the scale it is reporting both notes because both were in the window. The older of the two wins on level, having had more of the window to itself.

The solution adopted: the note-based modes ask a second question of one CQT frame, which carries no memory. A harmonic sum over the log-magnitude bins — the harmonic product spectrum, the axis being logarithmic — yields the pitch class sounding at that moment. It never named a class that was not played.

By default the estimate only **adds** a route to a pass, since overruling the model would forfeit the property that distinguishes this trainer from a monophonic one: the pitch head is polyphonic, so a strummed chord walks its intervals one by one. The **Play the notes one at a time** option makes the estimate the authority, and additionally requires a fresh attack before a repeated note counts a second time. Which attack counts is the subject of 8.13.

The residual latency is the 8192-sample FFT window, half a second wide, which is why notes shorter than approximately 0.4 s remain difficult. A shorter-window estimator in the time domain (autocorrelation) is the remaining avenue.

8.9. Input selection, and what a device list omits

A Windows machine reported no signal until the sample rate was changed by hand, which established that the sample format returned by the backend must not be discarded, and that the choice of device belongs to the user rather than to the operating system default.

The device list carries one property that is not obvious: **a card can be opened once.** Whatever holds it — a sound server, another application, or this application's own stream — removes it from the enumeration entirely. Three consequences follow, each of which was first observed as a defect:

- a list built after the stream opens is missing the card being recorded from,
- under PipeWire, which claims the hardware, only the four server names remain,
- the device being recorded from must be exempted from any "unavailable" marking, since its absence from the scan is the evidence that it is working.

The noise gate is stored per device. An interface and a laptop microphone sit tens of decibels apart, and a threshold that has to be found again after every switch is not a setting.

8.10. The cost of the application is one inference

`--bench` times a single inference. On the reference machine it takes 39 ms, and the model is asked every 40 ms, so the inference thread is saturated for as long as a chord rings; every other thread — rendering, the CQT, the audio callback — accounts for under 3% between them.

The same binary on the same machine under Windows reports 61 ms. The apparent tenfold discrepancy in reported load between the two systems proved to be a discrepancy between two counters rather than between two builds: `top` reports in units of one core and the Task Manager over the whole processor, so 100% of a core on eight cores is the same 12.5%.

8.11. Formulas, and a rule stricter than the one the note modes use

The application draws a set of intervals over a root — every subset of the twelve chromatic functions containing the root, 2048 in all — and the exercise is to find them on the neck and play them in any order. A function once credited stays credited for the lap, which changes what a false credit costs: in the other modes a wrong reading delays the exercise, here it removes a function from it permanently.

The rule was therefore measured rather than assumed. Over 49 notes of a real recording (`dist/latency_stats.py`):

rule	false credits	notes missed
all four paths at once, as the note modes use	110	0
the single-frame CQT estimate alone	33	0
the same, gated on the onset head	15	4

Of the 110, ninety-nine came from the model's pitch head — which answers "what is sounding", and a string ringing on or resonating in sympathy is sounding without having been played. Formulas therefore run on the single-frame estimate alone, with a vote of four of the last five audio frames; the onset gate was not adopted here, for the reason given in 8.12.

The same mode also plants a formula on a chord: its root is placed on one of the chord's twelve degrees, and how much of that chord the set then covers is counted — every tone of the chord except its root and its perfect fifth, since only those establish which chord it is. This is arithmetic over two twelve-bit masks and is exact, which is what makes it worth showing on screen beside the functions.

8.12. State carried across a boundary

The model answers about 0.77 s of audio. At the moment the exercise moves on — the next chord, the next lap, entry into a mode — its most recent answer is still about what came before that boundary. The application kept that answer, and credited the first target of the new chord from the ringing of the old one.

The symptom was reported as a property of the model: "it was better the first time", "recognition used to be faster". It was neither. The first chord after launch is clean because there is nothing to inherit; every chord after it starts holding the previous one. The correction is to discard what was heard at each such boundary — the pitch vector, the previous frame, the strike answer, the voting window and the last credit. Nothing is lost by discarding: the next audio frame is 16 ms away and the next inference 40 ms.

For a time the onset head was offered as an option — the model could credit only a class it also reported as struck. Once the boundary was corrected it was no longer needed for the reported symptom, and it carried a cost of its own: on the measurement above it removed 15 false credits at the price of 4 notes missed altogether. Since 0.5.10 the head is gone; "Credit only what was struck" gates on the strike detector instead (8.13).

8.13. Repeats: which string was struck

A class credited once may count again only when **that class** has been struck since — not when some string was hit, and not when the class is merely still sounding. Everything the application had answered one of those two easier questions: flux says a string was hit and is blind to which,

the ear and the voice count say which classes sound and are blind to whether they were just struck. Until 0.5.7 the rule was assembled from them, and the harness of Chapter 2 measured what that let through: a repeat allowed while the note merely rang, 24 times of 51.

The strike detector (5.4) answers the conjunction directly. Three things sit between it and the judge, each added because a measurement asked for it:

- **Levelling.** Its features are not level-invariant, and a quiet player reads as a weak attack: on the user's own capture, 11 dB under the recording it had been measured on, it found 42 of 87 notes. A slow gain, settling over about four seconds, brings the playing to the level the model knows: 81 of 87.
- **A refractory of 0.6 s per class.** The model also fires on decaying notes. In the exercises a class comes back only after a credit, the 0.35 s the finished set is shown for and the player's reply, so nothing real is lost.
- **An energy check on a class fired again within 2 s.** On three recordings of the user the model fired a class again within 2 s 37 times, in two groups with nothing between: 29 with the energy flat or falling — the note dying away — and 8 with it jumping 3.7 to 49 times — a string struck again. Such a re-fire waits 32 ms and counts only if the energy rose by a quarter. Waiting is the point: decided at the moment of firing, the same check refused real re-strikes, because the new note had barely entered the window.

measured on the 51 spliced notes	0.5.7	0.5.8
a repeat allowed while the note only rings	24	**0**
struck again after 0.8 s, counted in time	36, and 11 early	**48**, none early
struck again after 1.2 s, counted in time	35, and 14 early	**50**, none early
a note ringing into the next chord, credited there	47	**0**

The last row is a rule of its own. What is still sounding when the exercise moves to the next chord counts as already used there and needs a strike of its own — unless it was struck while the finished set was still shown, which is a player reaching the next chord early. Requiring a strike for every first credit would close the same leak but costs 7 to 11 of 87 notes the detector does not catch; carrying costs nothing where a note is played after it is asked for.

In Intervals, single notes, the fixed order and "only what was struck" are now on by default — together they are the measured path. Switched off all three, the mode becomes the other thing it can be: the model decides and nothing argues with it, carry-over included — chords and single chord tones played for fun rather than a test. Without the detector's file the application still starts and judges repeats on the older evidence, with its two largest leaks closed; that fallback lets 13 ringing notes of 51 through.

---

9. Hypotheses refuted by measurement

This chapter documents cases in which measurement refuted a previously held assumption.

hypothesis	measurement result
A normalisation discrepancy (per frame against global) constitutes the principal constraint	difference below 1 pp; the `InstanceNorm2d` layer compensates for it
A harmonic barrier — the third harmonic of b3 coincides with b7, rendering min7 and min indistinguishable	the measurement had been performed on segments carrying random labels
The model does not reproduce the training data (underfitting)	TRAIN check: 83.7% against 63.1% on validation, hence overfitting
Template matching will outperform the quality head (B ≈ 75% against A = 63%)	B = 61.0%
The 64% root ceiling follows from rootless jazz voicings	an artefact of the solo recordings; 97% in accompaniment
Masking the root will unblock quality beyond 73%	quality remained at 72%
The chroma in the distributed file is one-hot and therefore defective	`cq_to_chroma` at 24 bins per octave likewise assigns one weight per bin; the discrepancy concerned a shift
Without the `ORT_DYLIB_PATH` variable the binary will use the system library	`RUNPATH=$ORIGIN` from the `.cargo/config.toml` file already addressed this
The model has become worse at single notes	on isolated notes it places 0.96–0.99 on the correct class; on a scale its 0.77 s window credits the note before the one being played, in 79% of windows
A diminished seventh named from another of its notes is a different chord	it is the same four notes: C, Eb, Gb and A dim7 differ only in which one the model calls the root, and that follows the voicing rather than what was played
The onset head will make the better gate — it is the fastest answer available	on a recording it looked so: 202 ms against 676 ms. Applied live it refused far more than it caught, and on the crediting rule it traded 18 false credits for 4 notes missed
A third credited while the root was played is the fifth harmonic of that root	`--probe` over 364 windows: the false credits fall on +10 and +11 semitones, that is on the PREVIOUS note still inside the window, not on a harmonic
A single fresh answer from the onset head is too short a window to catch a strike	at a threshold of 0.02 the answer stays above the gate for a median of one second after the attack, and not one of 47 notes was left without a frame carrying it — the sixteen-frame memory kept for the purpose was removed
A strike model trained with ±6 dB of level spread does not care how loud the player is	on a capture 11 dB quieter it found 42 of 87 notes; levelled before the model, 81
A re-fire of the same class can be told from a decay by the energy at that moment	the model fires about 26 ms after the attack, while the window is still dominated by the old note; decided then, the check refused real restrikes. Decided 32 ms later, it separates them

hypothesis	measurement result
An octave jump in the pitch estimate means a new pluck	50 of the 58 false repeats of the older rule came from that one branch; it now needs an attack behind it

The pattern is unambiguous: **measurement results held consistently, whereas predictions formulated prior to measurement proved wrong in a systematic manner.** This justifies the adopted methodology based on probes.

---

10. Design decisions

10.1. Solutions adopted

The generator writes labels directly. No processing step reconstructs information from the signal.

Verification of labels by an independent script. A measure redundant with respect to the generator, applied deliberately.

Splitting the dataset by source. It lowers the reported figures by more than ten percentage points and is justified.

Four heads with separated roles. The note-based modes rely on the pitch vector rather than on the chord name; the fourth head answers for what was struck and is read, logged and offered as an option rather than being wired into the judging.

A separate network for strikes. Whether a ringing class was struck again is asked of a small causal network every 16 ms, not of the recognition model's 0.77 s window. It decides repeats and what carries across a chord; the trainer produces it in the file the application loads.

Two thresholds on the context window rather than one. The model is asked from half a window and its chord name believed from nine tenths — a single threshold cannot serve both a held chord and a single note.

Nothing heard before a boundary survives it. A chord change, a lap and a mode switch each discard the model's last answer, because that answer is about what came before them.

A sparse representation of the CQT kernel. The benefit is twofold: the size of the weights file and the processing time in the audio thread.

Rejection of incompatible weights by the application. Silent acceptance would result in a program that is functional but misclassifies.

Compiled-in strings, without the gettext library. With several dozen strings, a system dependency and `.mo` catalogues incur a cost exceeding the benefit.

10.2. Solutions rejected

Deriving quality from the pitch vector — measured as 21 percentage points inferior to the quality head.

Temporal aggregation as a means of improving quality — on a controlled population it yields approximately +1 pp. The model's errors are correlated in time: the model exhibits no vacillation between windows, but indicates the same incorrect answer consistently and with high confidence.

A bar-based mode (a score advancing in tempo, with assessment in place of a gate) — implemented and subsequently withdrawn. It was determined that the trainer is to respond dynamically.

Training phase 3 — measured as yielding no improvement across three runs.

10.3. Open matters

- **A test set from the target instrument.** All figures relate to six external performers and two renders of the synthetic dataset. No measurement on the target signal chain is available.
- **Changing `CTX_FRAMES` from 48 to 32** — no longer a free choice: the exported model fixes its input at 48 frames, so the change requires retraining. The latency it was intended to address was instead removed from the path where it mattered, by judging single notes on one CQT frame.
- **A pitch estimator with a shorter window.** Autocorrelation over roughly 100 ms would place the latency of a single note below the 512 ms FFT window, which is what still limits fast passages.
- **Increasing the quantity of material from a real instrument** — the only factor capable of reducing the 6.5 percentage point difference.
- **Quiet notes at the noise gate.** A fifth played at −60 dBFS, at the user's own gate, was not credited. The gate, not a crediting rule, is the lever, and no rule was changed for it.

---

11. Conclusion

The work carried out resulted in a model achieving 92.4% exact matches on validation determined with a split by source, released as distribution packages for two platforms.

The principal gain in accuracy followed not from changes to the architecture, but from four findings concerning the data:

1. half of the GuitarSet dataset consists of improvisation labelled with accompaniment chords,
2. the `instructed` annotation contains no sevenths and misclassifies five hundred segments,
3. splitting the dataset at the segment level introduces leakage,
4. pitch targets must be derived from the actual performance rather than from the score.

These four changes moved the `Exact` figure from 44.8% to 92.4%. None of them concerned the structure of the network.

The second network followed the same pattern. What made repeated notes honest was not a larger model but a question put precisely — which class was struck, not which is sounding — and measured on material where the answer is known: the reviewed notes of one recording, spliced into the situations the rules have to tell apart.

This document describes the state as of October 2026, version 0.5.8. Repository: https://github.com/greblus/solitito